

Guia de reciclaje de componentes web

Funcionalidades 41 a 50 - cierre del recurso unificado

HTML + CSS + JavaScript modular y version normal

Este documento continua el mismo formato de las guias anteriores. Aqui se cubren las funcionalidades restantes del recurso unificado, desde Todo list hasta Responsive design. La meta es que puedas reciclar cada funcionalidad sin depender del proyecto original completo.

El orden sigue la tabla de contenido del recurso unificado: 41 Todo list, 42 Drag and drop, 43 Conversor C/F/K, 44 Cambio de fondo, 45 Imprimir pagina, 46 Reset visual, 47 Helpers DOM/storage, 48 Version modular, 49 Version normal y 50 Responsive design.

Rango	Funcionalidades cubiertas	Objetivo de esta guia
41 a 50	Todo list, Drag and drop, Conversor C/F/K, Cambio de fondo, Imprimir pagina, Reset visual, Helpers DOM/storage, Version modular, Version normal y Responsive design.	Cerrar el banco de componentes con funcionalidades de productividad, estado visual, organizacion de JavaScript y adaptabilidad responsive.

Tabla de contenido

- 41. Todo list reutilizable
- 42. Drag and drop para reordenar tareas
- 43. Conversor C/F/K con botones de unidad
- 44. Cambio de fondo segun boton seleccionado
- 45. Imprimir pagina o seccion
- 46. Reset visual de estados y clases
- 47. Helpers DOM y storage
- 48. Version modular con export/import
- 49. Version normal sin modulos
- 50. Responsive design reutilizable

Como usar esta guia

- 1 Lee primero la descripcion de la funcionalidad y confirma que es lo que necesitas.
- 2 Copia el HTML minimo y cambia textos, ids y data-attributes.
- 3 Copia el CSS y adapta colores, espaciados, responsive y estados visuales.
- 4 Escoge una sola version de JavaScript: modular o normal.
- 5 Cambia los valores indicados en la tabla de variables de cada funcionalidad.
- 6 Prueba el checklist antes de reutilizar el componente en otro proyecto.

Regla importante: estas funcionalidades se pueden combinar. Por ejemplo, Todo list + Drag and drop + localStorage te permite crear una lista reordenable persistente. Conversor C/F/K + Cambio de fondo + Reset visual crea una mini app interactiva con estados visuales.

Convenciones que se repiten en las funcionalidades 41 a 50

Convencion	Para que sirve	Como adaptarla
storageKey	Clave unica para guardar datos en localStorage.	Usa nombres especificos como project-todos o sierra-notes para evitar mezclar datos.
data-id / data-unit / data-bg-class	Atributos HTML que comunican datos a JavaScript.	Cambia los valores, pero manten el mismo nombre si el JS los lee.
is-active / is-done / is-dragging	Clases de estado que activan estilos.	No las uses como clases base; JavaScript debe agregarlas o quitarlas.
callback	Funcion que se ejecuta cuando ocurre algo.	Usala para guardar cambios, mostrar toasts o conectar componentes.
@media	Reglas CSS para adaptar la vista a pantallas pequenas.	Ajusta breakpoints segun el diseno real.

Consejo de reutilizacion: cuando un componente usa ids, no repitas el mismo id mas de una vez. Si quieres dos TodoList en la misma pagina, cada una debe tener formId, inputId, listId y storageKey diferentes.

Mapa de combinaciones recomendadas

Combinacion	Para que sirve	Componentes involucrados
Todo list + Drag and drop	Lista de tareas reordenable con prioridades.	41 + 42 + 47
Conversor + Fondo dinamico + Reset	Calculadora interactiva con estados visuales y boton de reinicio.	43 + 44 + 46
Imprimir + Responsive	Reporte que se ve bien en pantalla y tambien en PDF/impresion.	45 + 50
Helpers + Version modular	Codigo mas limpio y reutilizable en proyectos grandes.	47 + 48
Version normal + Responsive	Proyecto simple sin modulos que aun se adapta a celular.	49 + 50

41. Todo list reutilizable

Aspecto	Explicacion
Que hace	Lista de tareas que permite crear, marcar como hecha y eliminar elementos. Es una base muy usada para paneles, checklists, tareas administrativas y prototipos CRUD.
Donde se usa en industria	Tableros de tareas, dashboards internos, seguimiento de pendientes, listas de compras, listas de validacion, checklists de onboarding y mini sistemas CRUD.
Como se personaliza	Cambia el id del formulario, input, lista, clave de localStorage, estructura de cada tarea y textos visibles. Puedes agregar prioridad, categoria, fecha limite o responsable.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section class="panel todo-widget" id="todoWidget">
  <h2>Lista de tareas</h2>

  <form class="todo-form" id="todoForm">
    <input id="todoInput" type="text" placeholder="Escribe una tarea" required>
    <button class="btn" type="submit">Agregar</button>
  </form>

  <ul class="todo-list" id="todoList"></ul>
</section>
```

CSS del componente

styles.css

```

.todo-widget {
  display: grid;
  gap: 16px;
}
.todo-form {
  display: grid;
  grid-template-columns: 1fr auto;
  gap: 10px;
}
.todo-form input {
  min-height: 44px;
  padding: 10px 12px;
  border-radius: 14px;
  border: 1px solid rgba(148, 163, 184, .5);
}
.todo-list {
  display: grid;
  gap: 10px;
  margin: 0;
  padding: 0;
  list-style: none;
}
.todo-item {
  display: grid;
  grid-template-columns: auto 1fr auto;
  align-items: center;
  gap: 10px;
  padding: 12px;
  border-radius: 16px;
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
}
.todo-item.is-done .todo-text {
  text-decoration: line-through;
  opacity: .55;
}
.todo-delete {
  border: 0;
  border-radius: 999px;
  padding: 8px 10px;
  cursor: pointer;
}

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class TodoList {
  constructor({ formId, inputId, listId, storageKey = 'kit-todos' }) {
    this.form = document.getElementById(formId);
    this.input = document.getElementById(inputId);
    this.list = document.getElementById(listId);
    this.storageKey = storageKey;
    this.todos = JSON.parse(localStorage.getItem(this.storageKey)) || [];
    this.init();
  }

  init() {
    this.form.addEventListener('submit', (event) => {
      event.preventDefault();
      this.add(this.input.value);
      this.input.value = '';
    });
    this.render();
  }

  save() {
    localStorage.setItem(this.storageKey, JSON.stringify(this.todos));
  }

  add(text) {
    const cleanText = text.trim();
    if (!cleanText) return;
    this.todos.push({ id: crypto.randomUUID(), text: cleanText, done: false });
    this.save();
    this.render();
  }

  toggle(id) {
    this.todos = this.todos.map((todo) => todo.id === id ? { ...todo, done: !todo.done } : todo);
    this.save();
    this.render();
  }

  remove(id) {
    this.todos = this.todos.filter((todo) => todo.id !== id);
    this.save();
    this.render();
  }

  render() {
    this.list.innerHTML = this.todos.map((todo) => `
      <li class="todo-item ${todo.done ? 'is-done' : ''}">
        <input type="checkbox" ${todo.done ? 'checked' : ''} data-toggle-todo="${todo.id}">
        <span class="todo-text">${todo.text}</span>
        <button class="todo-delete" type="button" data-delete-todo="${todo.id}">Eliminar</button>
      </li>
    `).join('');

    this.list.querySelectorAll('[data-toggle-todo]').forEach((checkbox) => {
      checkbox.addEventListener('change', () => this.toggle(checkbox.dataset.toggleTodo));
    });
    this.list.querySelectorAll('[data-delete-todo]').forEach((button) => {
      button.addEventListener('click', () => this.remove(button.dataset.deleteTodo));
    });
  }
}

// script.js
import { TodoList } from './funciones.js';
new TodoList({ formId: 'todoForm', inputId: 'todoInput', listId: 'todoList', storageKey: 'mis-tareas' });

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class TodoList {
  constructor(config) {
    this.form = document.getElementById(config.formId);
    this.input = document.getElementById(config.inputId);
    this.list = document.getElementById(config.listId);
    this.storageKey = config.storageKey || 'kit-todos';
    this.todos = JSON.parse(localStorage.getItem(this.storageKey)) || [];
    this.init();
  }
  init() {
    this.form.addEventListener('submit', (event) => {
      event.preventDefault();
      this.add(this.input.value);
      this.input.value = '';
    });
    this.render();
  }
  save() { localStorage.setItem(this.storageKey, JSON.stringify(this.todos)); }
  add(text) {
    const cleanText = text.trim();
    if (!cleanText) return;
    this.todos.push({ id: Date.now().toString(), text: cleanText, done: false });
    this.save();
    this.render();
  }
  toggle(id) {
    this.todos = this.todos.map((todo) => todo.id === id ? { ...todo, done: !todo.done } : todo);
    this.save();
    this.render();
  }
  remove(id) {
    this.todos = this.todos.filter((todo) => todo.id !== id);
    this.save();
    this.render();
  }
  render() {
    this.list.innerHTML = this.todos.map((todo) => `
      <li class="todo-item ${todo.done ? 'is-done' : ''}">
        <input type="checkbox" ${todo.done ? 'checked' : ''} data-toggle-todo="${todo.id}">
        <span class="todo-text">${todo.text}</span>
        <button class="todo-delete" type="button" data-delete-todo="${todo.id}">Eliminar</button>
      </li>
    `).join('');
    this.list.querySelectorAll('[data-toggle-todo]').forEach((el) => el.addEventListener('change', (
    ) => this.toggle(el.dataset.toggleTodo)));
    this.list.querySelectorAll('[data-delete-todo]').forEach((el) => el.addEventListener('click', (
    ) => this.remove(el.dataset.deleteTodo)));
  }
}
new TodoList({ formId: 'todoForm', inputId: 'todoInput', listId: 'todoList', storageKey: 'mis-tareas' });

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
todoForm	Formulario que captura la nueva tarea.	Cambia el id si tienes varios formularios.
todoInput	Input donde el usuario escribe.	Debe coincidir con el input real.
todoList	Contenedor donde se pintan las tareas.	Puede ser ul, div o section.
storageKey	Clave usada en localStorage.	Usa una clave unica por proyecto.
todo-item / is-done	Clases CSS de tarjeta y estado completado.	Cambia nombres si tu CSS usa otra convencion.

Implementacion paso a paso

- 1 Copia el HTML del widget donde quieras mostrar la lista.
- 2 Copia el CSS de la lista y ajusta colores o bordes segun tu diseno.
- 3 Copia la clase TodoList en funciones.js o en script-normal.js.
- 4 Inicializa la clase pasando los ids correctos.
- 5 Prueba agregar, marcar y eliminar una tarea.

6 Recarga la pagina para verificar que localStorage conserva los datos.

Errores comunes y como solucionarlos

- No se agrega la tarea: revisa que el formId y inputId sean correctos.
- Se duplican eventos: no inicialices TodoList dos veces con el mismo contenedor.
- Las tareas se pierden al recargar: revisa storageKey y permisos del navegador.
- El texto queda inseguro si aceptas HTML del usuario: en produccion conviene escapar el texto antes de pintar.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

42. Drag and drop para reordenar tareas

Aspecto	Explicacion
Que hace	Permite arrastrar elementos para cambiar su orden. Es muy util cuando el usuario necesita priorizar, ordenar columnas, mover cards o reorganizar listas.
Donde se usa en industria	Kanban, listas de tareas, gestores de prioridades, administradores de menus, orden de productos, dashboards y constructores visuales.
Como se personaliza	Cambia el selector de items, el contenedor, la clase visual al arrastrar y la funcion que guarda el nuevo orden. Si ya tienes TodoList, este componente se conecta al array de tareas.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<ul class="sortable-list" id="sortableList">
  <li class="sortable-item" draggable="true" data-id="1">Diseñar navbar</li>
  <li class="sortable-item" draggable="true" data-id="2">Crear sidebar</li>
  <li class="sortable-item" draggable="true" data-id="3">Validar formulario</li>
</ul>
```

CSS del componente

styles.css

```
.sortable-list {
  display: grid;
  gap: 10px;
  margin: 0;
  padding: 0;
  list-style: none;
}
.sortable-item {
  padding: 14px 16px;
  border-radius: 16px;
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
  cursor: grab;
  user-select: none;
  transition: transform .2s ease, opacity .2s ease, border-color .2s ease;
}
.sortable-item:active {
  cursor: grabbing;
}
.sortable-item.is-dragging {
  opacity: .45;
  transform: scale(.98);
  border-color: #60a5fa;
}
.sortable-item.drag-over {
  outline: 2px dashed #60a5fa;
  outline-offset: 4px;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```
export class DragSorter {
  constructor(listId, onChange = null) {
    this.list = document.getElementById(listId);
    this.onChange = onChange;
    this.dragging = null;
    this.init();
  }

  init() {
    this.list.addEventListener('dragstart', (event) => {
      const item = event.target.closest('[draggable="true"]');
      if (!item) return;
      this.dragging = item;
      item.classList.add('is-dragging');
    });

    this.list.addEventListener('dragend', () => {
      if (this.dragging) this.dragging.classList.remove('is-dragging');
      this.dragging = null;
      this.emitOrder();
    });

    this.list.addEventListener('dragover', (event) => {
      event.preventDefault();
      const afterElement = this.getAfterElement(event.clientY);
      if (!this.dragging) return;
      if (afterElement == null) this.list.appendChild(this.dragging);
      else this.list.insertBefore(this.dragging, afterElement);
    });
  }

  getAfterElement(y) {
    const items = [...this.list.querySelectorAll('[draggable="true"]:not(.is-dragging)')];
    return items.reduce((closest, child) => {
      const box = child.getBoundingClientRect();
      const offset = y - box.top - box.height / 2;
      if (offset < 0 && offset > closest.offset) return { offset, element: child };
      return closest;
    }, { offset: Number.NEGATIVE_INFINITY }).element;
  }

  emitOrder() {
    const order = [...this.list.children].map((item) => item.dataset.id);
    if (this.onChange) this.onChange(order);
  }
}

// script.js
import { DragSorter } from './funciones.js';
new DragSorter('sortableList', (order) => console.log('Nuevo orden:', order));
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como `script-normal.js` y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class DragSorter {
  constructor(listId, onChange) {
    this.list = document.getElementById(listId);
    this.onChange = onChange;
    this.dragging = null;
    this.init();
  }
  init() {
    this.list.addEventListener('dragstart', (event) => {
      const item = event.target.closest('[draggable="true"]');
      if (!item) return;
      this.dragging = item;
      item.classList.add('is-dragging');
    });
    this.list.addEventListener('dragend', () => {
      if (this.dragging) this.dragging.classList.remove('is-dragging');
      this.dragging = null;
      this.emitOrder();
    });
    this.list.addEventListener('dragover', (event) => {
      event.preventDefault();
      const afterElement = this.getAfterElement(event.clientY);
      if (!this.dragging) return;
      if (afterElement == null) this.list.appendChild(this.dragging);
      else this.list.insertBefore(this.dragging, afterElement);
    });
  }
  getAfterElement(y) {
    const items = [...this.list.querySelectorAll('[draggable="true"]:not(.is-dragging)')];
    return items.reduce((closest, child) => {
      const box = child.getBoundingClientRect();
      const offset = y - box.top - box.height / 2;
      return offset < 0 && offset > closest.offset ? { offset, element: child } : closest;
    }, { offset: Number.NEGATIVE_INFINITY }).element;
  }
  emitOrder() {
    const order = [...this.list.children].map((item) => item.dataset.id);
    if (this.onChange) this.onChange(order);
  }
}
new DragSorter('sortableList', (order) => console.log('Nuevo orden:', order));

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
sortableList	Id del contenedor que contiene los elementos arrastrables.	Debe coincidir con el ul o div real.
draggable="true"	Activa la capacidad de arrastrar un elemento.	Debe ir en cada item.
data-id	Identificador real de cada item.	Usa ids de base de datos o ids internos.
is-dragging	Clase visual aplicada durante el arrastre.	Puedes cambiar opacidad, borde o escala.
onChange(order)	Callback que recibe el nuevo orden.	Usalo para guardar en localStorage o enviar al backend.

Implementacion paso a paso

1. Agrega draggable="true" a cada item que quieras mover.
2. Agrega data-id para reconocer cada elemento despues de moverlo.
3. Copia el CSS del estado normal y del estado is-dragging.
4. Copia DragSorter y pasale el id del contenedor.
5. En el callback onChange guarda el nuevo orden si lo necesitas.
6. Prueba mover elementos hacia arriba y hacia abajo.

Errores comunes y como solucionarlos

- No se mueve nada: falta `event.preventDefault()` en `dragover`.
- Se pierde el orden al recargar: debes guardar el array `order` en `localStorage` o backend.
- En movil no funciona igual: `drag and drop` nativo en `touch` puede requerir libreria o eventos `pointer`.
- Todos los items tienen el mismo `data-id`: usa identificadores unicos.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un `Toast` para avisar acciones exitosas o errores.
- Guardar preferencias o datos en `localStorage` cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con `aria-label`, `aria-live`, `focus-visible` y navegacion por teclado cuando aplique.

43. Conversor C/F/K con botones de unidad

Aspecto	Explicacion
Que hace	Herramienta donde el usuario elige la unidad con botones C, F o K y luego calcula las conversiones restantes. Sirve como patron para apps donde un boton controla la logica.
Donde se usa en industria	Calculadoras, conversores, filtros rapidos, configuradores, formularios con opciones de modo, dashboards con cambio de metrica y herramientas educativas.
Como se personaliza	Cambia los botones data-unit, formulas, textos del resultado, ids del input y contenedor. Puedes reemplazar temperatura por moneda, medidas, porcentajes o cualquier calculo.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section class="panel temperature-tool" id="temperatureTool">
  <h2>Conversor de temperatura</h2>

  <div class="unit-buttons" role="group" aria-label="Unidad inicial">
    <button class="unit-button is-active" type="button" data-unit="celsius">C</button>
    <button class="unit-button" type="button" data-unit="fahrenheit">F</button>
    <button class="unit-button" type="button" data-unit="kelvin">K</button>
  </div>

  <input id="temperatureInput" type="number" placeholder="Ingrese temperatura">
  <button class="btn" id="temperatureCalculate" type="button">Calcular</button>
  <p class="message" id="temperatureMessage"></p>
  <div class="result-box" id="temperatureResult"></div>
</section>
```

CSS del componente

styles.css

```
.temperature-tool {
  display: grid;
  gap: 16px;
}
.unit-buttons {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
.unit-button {
  min-height: 46px;
  border: 1px solid rgba(255,255,255,.25);
  border-radius: 16px;
  color: #fff;
  background: rgba(255,255,255,.12);
  cursor: pointer;
  font-weight: 800;
  transition: transform .2s ease, background .2s ease;
}
.unit-button:hover,
.unit-button.is-active {
  transform: translateY(-2px);
  background: linear-gradient(135deg, #2563eb, #7c3aed);
}
.result-box {
  padding: 16px;
  border-radius: 18px;
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
  line-height: 1.7;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```

export class TemperatureTool {
  constructor(config) {
    this.buttons = document.querySelectorAll(config.buttonSelector);
    this.input = document.getElementById(config.inputId);
    this.calculateButton = document.getElementById(config.calculateId);
    this.message = document.getElementById(config.messageId);
    this.result = document.getElementById(config.resultId);
    this.unit = 'celsius';
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener('click', () => this.selectUnit(button));
    });
    this.calculateButton.addEventListener('click', () => this.calculate());
  }

  selectUnit(button) {
    this.unit = button.dataset.unit;
    this.buttons.forEach((item) => item.classList.remove('is-active'));
    button.classList.add('is-active');
  }

  calculate() {
    const value = Number(this.input.value.trim());
    if (!Number.isFinite(value)) {
      this.message.textContent = 'Ingresa un numero valido.';
      this.result.innerHTML = '';
      return;
    }
    this.message.textContent = 'Calculo realizado.';

    if (this.unit === 'celsius') {
      this.result.innerHTML = `Fahrenheit: ${((value * 9 / 5) + 32).toFixed(2)} F<br>Kelvin: ${((
value + 273.15).toFixed(2))} K`;
    }
    if (this.unit === 'fahrenheit') {
      const c = (value - 32) * 5 / 9;
      this.result.innerHTML = `Celsius: ${c.toFixed(2)} C<br>Kelvin: ${((c + 273.15).toFixed(2))} K`;
    }
    if (this.unit === 'kelvin') {
      const c = value - 273.15;
      this.result.innerHTML = `Celsius: ${c.toFixed(2)} C<br>Fahrenheit: ${(((c * 9 / 5) + 32).
toFixed(2))} F`;
    }
  }
}

// script.js
import { TemperatureTool } from './funciones.js';
new TemperatureTool({
  buttonSelector: '.unit-button', inputId: 'temperatureInput', calculateId: 'temperatureCalculate',
  messageId: 'temperatureMessage', resultId: 'temperatureResult'
});

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js


```

class TemperatureTool {
  constructor(config) {
    this.buttons = document.querySelectorAll(config.buttonSelector);
    this.input = document.getElementById(config.inputId);
    this.calculateButton = document.getElementById(config.calculateId);
    this.message = document.getElementById(config.messageId);
    this.result = document.getElementById(config.resultId);
    this.unit = 'celsius';
    this.init();
  }
  init() {
    this.buttons.forEach((button) => button.addEventListener('click', () => this.selectUnit(
button)));
    this.calculateButton.addEventListener('click', () => this.calculate());
  }
  selectUnit(button) {
    this.unit = button.dataset.unit;
    this.buttons.forEach((item) => item.classList.remove('is-active'));
    button.classList.add('is-active');
  }
  calculate() {
    const value = Number(this.input.value.trim());
    if (!Number.isFinite(value)) {
      this.message.textContent = 'Ingresa un numero valido.';
      this.result.innerHTML = '';
      return;
    }
    this.message.textContent = 'Calculo realizado.';
    if (this.unit === 'celsius') this.result.innerHTML = `Fahrenheit: ${((value * 9 / 5) + 32).
toFixed(2)} F<br>Kelvin: ${((value + 273.15).toFixed(2))} K`;
    if (this.unit === 'fahrenheit') {
      const c = (value - 32) * 5 / 9;
      this.result.innerHTML = `Celsius: ${c.toFixed(2)} C<br>Kelvin: ${((c + 273.15).toFixed(2))} K`;
    }
    if (this.unit === 'kelvin') {
      const c = value - 273.15;
      this.result.innerHTML = `Celsius: ${c.toFixed(2)} C<br>Fahrenheit: ${(((c * 9 / 5) + 32).
toFixed(2))} F`;
    }
  }
}
new TemperatureTool({ buttonSelector: '.unit-button', inputId: 'temperatureInput', calculateId:
'temperatureCalculate', messageId: 'temperatureMessage', resultId: 'temperatureResult' });

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data-unit	Define la unidad seleccionada por cada boton.	Valores esperados: celsius, fahrenheit, kelvin.
unit-button	Clase de los botones de unidad.	Usala para estilos y seleccion con querySelectorAll.
temperatureInput	Campo numerico principal.	Cambia si tu calculadora usa otro input.
temperatureResult	Caja donde se imprime el resultado.	Puedes usar innerHTML para multiples lineas.
this.unit	Estado actual de la unidad seleccionada.	Puedes iniciar en fahrenheit o kelvin si quieres.

Implementacion paso a paso

- 1 Copia el HTML de los tres botones y el input.
- 2 Asegurate de que cada boton tenga data-unit diferente.
- 3 Copia el CSS de unit-buttons, unit-button y result-box.
- 4 Copia la clase TemperatureTool y cambiale los ids en la instancia.
- 5 Prueba cada unidad con un valor conocido.
- 6 Si lo reciclas para otro calculo, reemplaza las formulas dentro de calculate().

Errores comunes y como solucionarlos

- Siempre calcula Celsius: revisa que data-unit este bien escrito.
- El boton no se activa visualmente: revisa la clase is-active.
- El resultado dice NaN: valida el input antes de calcular.
- innerHTML no aparece: revisa que resultId coincida con el HTML.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

44. Cambio de fondo segun boton seleccionado

Aspecto	Explicacion
Que hace	Cambia el estado visual global de la pagina agregando clases al body. En el ejemplo, C coloca fondo rojo, F azul y K verde.
Donde se usa en industria	Temas por categoria, pasos de un wizard, modo de visualizacion, cambio de marca, estados de calculadora, filtros destacados y pantallas interactivas.
Como se personaliza	Cambia las clases body.unit-celsius, body.unit-fahrenheit y body.unit-kelvin. Puedes usar cualquier nombre: mode-admin, theme-gold, state-success, etc.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<div class="unit-buttons" role="group" aria-label="Cambiar fondo">
  <button class="unit-button" type="button" data-bg-class="unit-celsius">Fondo C</button>
  <button class="unit-button" type="button" data-bg-class="unit-fahrenheit">Fondo F</button>
  <button class="unit-button" type="button" data-bg-class="unit-kelvin">Fondo K</button>
</div>
```

CSS del componente

styles.css

```
body.unit-celsius {
  background: radial-gradient(circle at top left, #ef4444, transparent 35%),
    linear-gradient(135deg, #450a0a, #111827);
}
body.unit-fahrenheit {
  background: radial-gradient(circle at top left, #3b82f6, transparent 35%),
    linear-gradient(135deg, #082f49, #111827);
}
body.unit-kelvin {
  background: radial-gradient(circle at top left, #22c55e, transparent 35%),
    linear-gradient(135deg, #052e16, #111827);
}
body.unit-celsius .unit-button[data-bg-class="unit-celsius"],
body.unit-fahrenheit .unit-button[data-bg-class="unit-fahrenheit"],
body.unit-kelvin .unit-button[data-bg-class="unit-kelvin"] {
  background: rgba(255,255,255,.28);
  border-color: rgba(255,255,255,.75);
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class BodyClassSwitcher {
  constructor(buttonSelector, allowedClasses) {
    this.buttons = document.querySelectorAll(buttonSelector);
    this.allowedClasses = allowedClasses;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener('click', () => {
        this.setClass(button.dataset.bgClass);
      });
    });
  }

  setClass(className) {
    document.body.classList.remove(...this.allowedClasses);
    if (this.allowedClasses.includes(className)) {
      document.body.classList.add(className);
    }
  }
}

// script.js
import { BodyClassSwitcher } from './funciones.js';
new BodyClassSwitcher('[data-bg-class]', ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin']);
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class BodyClassSwitcher {
  constructor(buttonSelector, allowedClasses) {
    this.buttons = document.querySelectorAll(buttonSelector);
    this.allowedClasses = allowedClasses;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener('click', () => this.setClass(button.dataset.bgClass));
    });
  }

  setClass(className) {
    document.body.classList.remove(...this.allowedClasses);
    if (this.allowedClasses.includes(className)) document.body.classList.add(className);
  }
}

new BodyClassSwitcher('[data-bg-class]', ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin']);
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data-bg-class	Clase que el boton quiere poner en body.	Debe existir en el CSS.
allowedClasses	Lista de clases permitidas.	Agrega todas las clases que se deben limpiar antes de cambiar.
document.body.classList	Controla clases globales del body.	No lo cambies salvo que quieras afectar otro contenedor.
body.unit-*	Reglas CSS que cambian fondo.	Cambia degradados, colores e imagenes.

Implementacion paso a paso

- 1 Define que clases visuales quieres usar en body.
- 2 Crea un boton por cada clase usando data-bg-class.

- 3 Copia las reglas CSS `body.nombreClase`.
- 4 Inicializa `BodyClassSwitcher` con el selector y array de clases permitidas.
- 5 Haz clic en cada boton y revisa que el body solo tenga una clase visual activa.

Errores comunes y como solucionarlos

- Se mezclan fondos: olvidaste quitar clases anteriores con `classList.remove(...allowedClasses)`.
- No cambia el fondo: el valor `data-bg-class` no coincide con la clase CSS.
- El cambio afecta toda la pagina: si quieres solo una seccion, aplica la clase a un contenedor en vez de `body`.
- El texto pierde contraste: ajusta colores de texto para cada fondo.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en `localStorage` cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con `aria-label`, `aria-live`, `focus-visible` y navegacion por teclado cuando aplique.

45. Imprimir pagina o seccion

Aspecto	Explicacion
Que hace	Abre el dialogo de impresion del navegador. Es util para facturas, reportes, comprobantes, ordenes, hojas de vida, certificados o resúmenes.
Donde se usa en industria	Paneles administrativos, recibos, reportes internos, tickets, comprobantes de pedido, documentos de soporte y paginas de resumen.
Como se personaliza	Puedes imprimir toda la pagina con <code>window.print()</code> o preparar CSS @media print para ocultar botones, navbar y elementos que no deben aparecer.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section class="print-area" id="invoiceArea">
  <h2>Resumen imprimible</h2>
  <p>Cliente: Carlos Madero</p>
  <p>Total: $120.000</p>
</section>

<button class="btn no-print" id="printPage" type="button">Imprimir</button>
```

CSS del componente

styles.css

```
.print-area {
  padding: 24px;
  border-radius: 18px;
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
}
@media print {
  body {
    background: #ffffff !important;
    color: #111827 !important;
  }
  .no-print,
  .navbar,
  .sidebar,
  .toast-container,
  .modal {
    display: none !important;
  }
  .print-area {
    background: #ffffff !important;
    color: #111827 !important;
    border: 1px solid #d1d5db !important;
    box-shadow: none !important;
  }
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class PrintButton {
  constructor(buttonId, beforePrint = null) {
    this.button = document.getElementById(buttonId);
    this.beforePrint = beforePrint;
    this.init();
  }

  init() {
    this.button.addEventListener('click', () => {
      if (this.beforePrint) this.beforePrint();
      window.print();
    });
  }
}

// script.js
import { PrintButton } from './funciones.js';
new PrintButton('printPage', () => console.log('Preparando impresion...'));
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class PrintButton {
  constructor(buttonId, beforePrint) {
    this.button = document.getElementById(buttonId);
    this.beforePrint = beforePrint;
    this.init();
  }

  init() {
    this.button.addEventListener('click', () => {
      if (this.beforePrint) this.beforePrint();
      window.print();
    });
  }
}

new PrintButton('printPage', () => console.log('Preparando impresion...'));
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
printPage	Boton que dispara la impresion.	Cambia el id si tienes varios botones.
no-print	Clase para ocultar elementos al imprimir.	Agregala a botones, menus o elementos decorativos.
@media print	CSS especial para impresion.	Ajusta colores, margenes y elementos visibles.
beforePrint	Callback antes de imprimir.	Usalo para calcular totales o cerrar modales.

Implementacion paso a paso

- 1 Agrega el boton de imprimir con id unico.
- 2 Agrega clase no-print a todo lo que no debe salir en papel.
- 3 Copia @media print en tu CSS.
- 4 Inicializa PrintButton en tu JS.
- 5 Prueba vista previa de impresion y ajusta margenes o colores.

Errores comunes y como solucionarlos

- Se imprime el menu: agrega no-print o regla display none en @media print.
- Los colores salen oscuros: redefine body y print-area en @media print.
- El boton no responde: revisa que el id printPage coincida.
- Quieres imprimir solo una seccion: debes usar CSS print para ocultar todo excepto esa seccion.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

46. Reset visual de estados y clases

Aspecto	Explicacion
Que hace	Limpia clases temporales de la interfaz para regresar a un estado base. Es util cuando muchos botones cambian fondos, temas, filtros o estados visuales.
Donde se usa en industria	Dashboards con filtros, calculadoras, formularios multi-paso, juegos sencillos, prototipos de UI, reset de demos y limpieza de estados visuales.
Como se personaliza	Cambia el array de clases a limpiar, el boton de reset y si tambien quieres limpiar inputs, resultados o localStorage.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="btn" id="resetVisual" type="button">Reset visual</button>  
<p id="resetMessage" class="message"></p>
```

CSS del componente

styles.css

```
.message {  
  min-height: 20px;  
  color: rgba(255,255,255,.8);  
}  
body.unit-celsius,  
body.unit-fahrenheit,  
body.unit-kelvin {  
  transition: background .35s ease, color .35s ease;  
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class VisualReset {
  constructor(config) {
    this.button = document.getElementById(config.buttonId);
    this.message = document.getElementById(config.messageId);
    this.target = config.target || document.body;
    this.classesToRemove = config.classesToRemove;
    this.extraReset = config.extraReset || null;
    this.init();
  }

  init() {
    this.button.addEventListener('click', () => this.reset());
  }

  reset() {
    this.target.classList.remove(...this.classesToRemove);
    if (this.extraReset) this.extraReset();
    if (this.message) this.message.textContent = 'Vista reiniciada.';
  }
}

// script.js
import { VisualReset } from './funciones.js';
new VisualReset({
  buttonId: 'resetVisual',
  messageId: 'resetMessage',
  classesToRemove: ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin']
});
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class VisualReset {
  constructor(config) {
    this.button = document.getElementById(config.buttonId);
    this.message = document.getElementById(config.messageId);
    this.target = config.target || document.body;
    this.classesToRemove = config.classesToRemove;
    this.extraReset = config.extraReset || null;
    this.init();
  }
  init() { this.button.addEventListener('click', () => this.reset()); }
  reset() {
    this.target.classList.remove(...this.classesToRemove);
    if (this.extraReset) this.extraReset();
    if (this.message) this.message.textContent = 'Vista reiniciada.';
  }
}
new VisualReset({ buttonId: 'resetVisual', messageId: 'resetMessage', classesToRemove: ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin'] });
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
buttonId	Boton que ejecuta el reset.	Debe coincidir con el id HTML.
classesToRemove	Array con clases que se limpian.	Agrega aqui todos los estados visuales temporales.
target	Elemento al que se le quitan clases.	Por defecto es document.body.
extraReset	Funcion opcional para limpiar inputs o resultados.	Usala si quieres resetear mas cosas.

Implementacion paso a paso

- 1 Identifica las clases visuales temporales del proyecto.
- 2 Crea el boton de reset.

- 3 Copia la clase VisualReset.
- 4 Pasa el array de clases en classesToRemove.
- 5 Si necesitas limpiar campos, agrega extraReset.
- 6 Prueba activar varios estados y luego reiniciarlos.

Errores comunes y como solucionarlos

- No limpia nada: la clase en classesToRemove no coincide con la clase real.
- Limpia demasiado: no metas clases base del layout en el array.
- El mensaje no aparece: messageId no coincide o el elemento no existe.
- El reset visual no borra datos: solo borra clases; usa extraReset para datos.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

47. Helpers DOM y storage

Aspecto	Explicacion
Que hace	Pequeñas funciones reutilizables para escribir menos código repetido al seleccionar elementos, guardar JSON, leer JSON, mostrar mensajes o formatear datos.
Donde se usa en industria	Cualquier proyecto frontend: landing pages, formularios, dashboards, e-commerce, proyectos académicos, prototipos y aplicaciones SPA simples.
Como se personaliza	Puedes agregar helpers nuevos según tu estilo: validarEmail, formatCurrency, createElement, showError, clearHTML, debounce, throttle, etc.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML mínimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interacción principal.

HTML mínimo que debes copiar

index.html

```
<p id="helperMessage"></p>
<button class="btn" id="helperDemo" type="button">Probar helper</button>
```

CSS del componente

styles.css

```
.helper-ok {
  color: #22c55e;
  font-weight: 700;
}
.helper-error {
  color: #ef4444;
  font-weight: 700;
}
```

JavaScript modular con export/import

Usa esta versión si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o función vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```

export const $ = (selector, parent = document) => parent.querySelector(selector);
export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch (error) {
    console.error('Error leyendo JSON:', error);
    return fallback;
  }
}

export function showMessage(element, text, type = 'ok') {
  element.textContent = text;
  element.classList.remove('helper-ok', 'helper-error');
  element.classList.add(type === 'ok' ? 'helper-ok' : 'helper-error');
}

export function formatMoney(value, locale = 'es-CO', currency = 'COP') {
  return new Intl.NumberFormat(locale, { style: 'currency', currency }).format(value);
}

// script.js
import { $, saveJSON, readJSON, showMessage, formatMoney } from './funciones.js';
$('#helperDemo').addEventListener('click', () => {
  saveJSON('demo-price', { price: 12000 });
  const data = readJSON('demo-price', { price: 0 });
  showMessage($('#helperMessage'), `Precio: ${formatMoney(data.price)}`, 'ok');
});

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}
function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch (error) {
    console.error('Error leyendo JSON:', error);
    return fallback;
  }
}
function showMessage(element, text, type = 'ok') {
  element.textContent = text;
  element.classList.remove('helper-ok', 'helper-error');
  element.classList.add(type === 'ok' ? 'helper-ok' : 'helper-error');
}
function formatMoney(value, locale = 'es-CO', currency = 'COP') {
  return new Intl.NumberFormat(locale, { style: 'currency', currency }).format(value);
}
$('#helperDemo').addEventListener('click', () => {
  saveJSON('demo-price', { price: 12000 });
  const data = readJSON('demo-price', { price: 0 });
  showMessage($('#helperMessage'), `Precio: ${formatMoney(data.price)}`, 'ok');
});

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
\$	Selecciona el primer elemento que coincide.	Usalo como atajo de querySelector.

Elemento	Para que sirve	Que debes cambiar
\$\$	Selecciona varios elementos y los devuelve array.	Util para forEach, map, filter.
saveJSON / readJSON	Guarda y lee objetos en localStorage.	Cambia la key segun el proyecto.
showMessage	Muestra texto y aplica clase de estado.	Cambia clases helper-ok/helper-error.
formatMoney	Formatea dinero segun pais y moneda.	Cambia locale y currency.

Implementacion paso a paso

- 1 Copia los helpers al inicio de funciones.js o script-normal.js.
- 2 Exportalos si trabajas modular.
- 3 Usalos en tus componentes para evitar repetir codigo.
- 4 Agrega nuevos helpers cuando una accion se repita varias veces.
- 5 No crees helpers demasiado especificos si solo los usaras una vez.

Errores comunes y como solucionarlos

- \$ no existe: en modular debes importarlo antes de usarlo.
- readJSON falla: el contenido guardado no es JSON valido.
- localStorage no guarda objetos directamente: usa JSON.stringify como en saveJSON.
- formatMoney muestra otra moneda: cambia locale y currency.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

48. Version modular con export/import

Aspecto	Explicacion
Que hace	Organiza el codigo en archivos separados. funciones.js contiene clases o utilidades exportadas y script.js importa lo necesario para iniciar la pagina.
Donde se usa en industria	Frontend moderno, proyectos escalables, equipos de desarrollo, componentes reutilizables, separacion de responsabilidades y mantenimiento limpio.
Como se personaliza	Cambia los nombres de clases, rutas de importacion y funciones exportadas. Recuerda cargar el script principal con type="module".

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<!-- index.html -->
<script type="module" src="script.js"></script>
```

CSS del componente

styles.css

```
/* CSS no cambia por usar modulos.
   La modularidad es una forma de organizar JavaScript. */
.app {
  min-height: 100vh;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga <script type="module" src="script.js"></script>. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
// funciones.js
export class SaludoApp {
  constructor(messageId) {
    this.message = document.getElementById(messageId);
  }

  show(name) {
    this.message.textContent = `Hola, ${name}`;
  }
}

export function cleanText(value) {
  return value.trim();
}

// script.js
import { SaludoApp, cleanText } from './funciones.js';

const app = new SaludoApp('mensaje');
const nombre = cleanText(' Carlos ');
app.show(nombre);
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
// En version normal no usas export ni import.
// Todo queda disponible en el mismo archivo script-normal.js.
class SaludoApp {
  constructor(messageId) {
    this.message = document.getElementById(messageId);
  }
  show(name) {
    this.message.textContent = `Hola, ${name}`;
  }
}
function cleanText(value) {
  return value.trim();
}
const app = new SaludoApp('mensaje');
const nombre = cleanText(' Carlos ');
app.show(nombre);
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
type="module"	Activa soporte de import/export en el navegador.	Debe ir en el script principal.
export	Permite sacar una clase o funcion de un archivo.	Ponlo delante de class, function o const.
import	Trae codigo desde otro archivo.	La ruta debe empezar con ./ si es archivo local.
./funciones.js	Ruta del archivo que exporta.	Cambia si tu archivo se llama distinto.
script.js	Archivo que arranca la aplicacion.	Debe importar y crear instancias.

Implementacion paso a paso

- 1 Crea funciones.js para clases y funciones reutilizables.
- 2 Exporta cada clase o funcion que necesites usar afuera.
- 3 Crea script.js como archivo de arranque.
- 4 Importa lo necesario desde funciones.js.

- 5 En HTML carga script.js con type="module".
- 6 Ejecuta con Live Server; no abras el HTML como archivo local si hay problemas de modulo.

Errores comunes y como solucionarlos

- Cannot use import statement outside a module: falta type="module".
- Failed to resolve module specifier: la ruta del import esta mal.
- No se encuentra una clase: falta exportarla o importarla con el mismo nombre.
- Se ejecuta antes del HTML: coloca el script al final del body o usa DOMContentLoaded.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

49. Version normal sin modulos

Aspecto	Explicacion
Que hace	Permite usar JavaScript en un solo archivo sin export/import. Es util para ejercicios simples, entregas basicas o proyectos donde no quieres configurar modulos.
Donde se usa en industria	Landing pages pequenas, ejercicios academicos, scripts de una sola pagina, prototipos rapidos, integraciones muy simples y demos.
Como se personaliza	Cambia el nombre del archivo script-normal.js y el orden de carga. Si separas varios scripts normales, el orden importa mucho.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<!-- index.html -->
<script src="script-normal.js"></script>
```

CSS del componente

styles.css

```
/* CSS tampoco cambia por usar version normal. */
.page-ready {
  opacity: 1;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
// Equivalencia conceptual en modular:
// funciones.js
export class AppNormalEquivalente {
  start() {
    console.log('App iniciada en modo modular');
  }
}

// script.js
import { AppNormalEquivalente } from './funciones.js';
new AppNormalEquivalente().start();
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con <script src="script-normal.js"></script>.

script-normal.js

```
// script-normal.js
class AppNormal {
  start() {
    console.log('App iniciada sin modulos');
  }
}

function initApp() {
  const app = new AppNormal();
  app.start();
}

// Opcion 1: si el script esta al final del body
initApp();

// Opcion 2: si el script esta en el head
// document.addEventListener('DOMContentLoaded', initApp);
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
script-normal.js	Archivo unico donde vive todo el codigo.	Puedes cambiar el nombre, pero actualiza el src.
DOMContentLoaded	Evento que espera a que cargue el HTML.	Usalo si el script esta en head.
orden de scripts	En JS normal, lo de arriba debe existir antes de usarse.	Carga helpers antes que componentes que los usan.
variables globales	Todo puede quedar expuesto en window si usas var.	Prefiere const, let y clases para evitar choques.

Implementacion paso a paso

- 1 Crea un archivo script-normal.js.
- 2 Pega helpers, clases y luego el codigo de inicializacion.
- 3 Carga el script al final del body.
- 4 No uses import ni export en este archivo.
- 5 Si tienes varios archivos normales, carga primero helpers, despues clases, despues script principal.

Errores comunes y como solucionarlos

- Unexpected token export: estas usando export en un script normal.
- Clase no definida: cargaste los scripts en mal orden.
- document.getElementById retorna null: el script se ejecuto antes de que exista el HTML.
- Choques de nombres: evita variables globales genericas como data, item, app si hay muchos scripts.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.

- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

50. Responsive design reutilizable

Aspecto	Explicacion
Que hace	Hace que la pagina se adapte a celular, tablet y escritorio. Es una de las partes mas importantes para que una web se vea profesional.
Donde se usa en industria	Todas las paginas web modernas: landings, portafolios, ecommerce, dashboards, formularios, blogs, sistemas internos y aplicaciones web.
Como se personaliza	Cambia breakpoints, grid, tamanos de fuente, padding, columnas y comportamiento de navbar/sidebar. Prueba siempre en menos de 520px.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<main class="responsive-layout">
  <section class="hero-card">Contenido principal</section>
  <aside class="side-card">Contenido secundario</aside>
</main>
```

CSS del componente

styles.css

```

:root {
  --container: 1120px;
  --space: 24px;
  --radius: 22px;
}
.responsive-layout {
  width: min(var(--container), calc(100% - 32px));
  margin: 0 auto;
  display: grid;
  grid-template-columns: minmax(0, 2fr) minmax(280px, 1fr);
  gap: var(--space);
}
.hero-card,
.side-card {
  padding: 24px;
  border-radius: var(--radius);
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
}
@media (max-width: 900px) {
  .responsive-layout {
    grid-template-columns: 1fr;
  }
}
@media (max-width: 520px) {
  :root {
    --space: 14px;
  }
  .responsive-layout {
    width: min(100% - 20px, var(--container));
  }
  .hero-card,
  .side-card {
    padding: 18px;
    border-radius: 16px;
  }
  h1 {
    font-size: clamp(1.7rem, 9vw, 2.4rem);
  }
}

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class ResponsiveHelper {
  constructor(outputId) {
    this.output = document.getElementById(outputId);
    this.init();
  }

  init() {
    this.update();
    window.addEventListener('resize', () => this.update());
  }

  update() {
    const width = window.innerWidth;
    let label = 'desktop';
    if (width <= 520) label = 'mobile';
    else if (width <= 900) label = 'tablet';
    if (this.output) this.output.textContent = `Vista actual: ${label} (${width}px)`;
    document.body.dataset.viewport = label;
  }
}

// script.js
import { ResponsiveHelper } from './funciones.js';
new ResponsiveHelper('viewportLabel');

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class ResponsiveHelper {
  constructor(outputId) {
    this.output = document.getElementById(outputId);
    this.init();
  }
  init() {
    this.update();
    window.addEventListener('resize', () => this.update());
  }
  update() {
    const width = window.innerWidth;
    let label = 'desktop';
    if (width <= 520) label = 'mobile';
    else if (width <= 900) label = 'tablet';
    if (this.output) this.output.textContent = `Vista actual: ${label} (${width}px)`;
    document.body.dataset.viewport = label;
  }
}
new ResponsiveHelper('viewportLabel');

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
--container	Ancho maximo del contenido.	Cambia segun diseno: 960px, 1120px, 1280px.
breakpoints	Puntos donde cambia el layout.	Ajusta 900px y 520px segun tu pagina.
grid-template-columns	Define columnas en desktop.	Cambia 2fr/1fr por 1fr/1fr o repeat.
clamp()	Controla tamanos fluidos.	Util para titulos responsive.
data-viewport	Etiqueta el estado actual desde JS.	Puedes usarla para depurar o cambiar estilos.

Implementacion paso a paso

- 1 Empieza disenando mobile o desktop, pero prueba ambos.
- 2 Usa contenedores con width: min(...) para evitar desbordes.
- 3 Convierte grids de varias columnas a una columna en tablet/mobile.
- 4 Reduce padding y radios en pantallas pequenas.
- 5 Prueba navbar, sidebar, formularios, modales y tablas en menos de 520px.
- 6 Usa DevTools para revisar distintos anchos.

Errores comunes y como solucionarlos

- Aparece scroll horizontal: revisa widths fijos, imagenes grandes o grid sin minmax(0, 1fr).
- El texto queda gigante en movil: usa clamp o media queries.
- Las tablas se rompen: agrega overflow-x:auto al contenedor.
- El sidebar tapa todo: define comportamiento mobile para menus fijos.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.

- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

Cierre: como usar las funcionalidades 41 a 50 en proyectos reales

Estas ultimas funcionalidades completan el banco porque cubren tareas muy comunes: guardar tareas, reordenar elementos, calcular valores, cambiar estados visuales, imprimir, reiniciar interfaces, usar helpers, decidir entre JS modular o normal y hacer responsive el proyecto. Si entiendes estas piezas, puedes armar muchas paginas profesionales sin empezar desde cero.

Patron recomendado para un proyecto reutilizable

Ejemplo de integracion modular

```
// 1. HTML define estructura e ids.
// 2. CSS define apariencia y estados visuales.
// 3. funciones.js guarda clases reutilizables.
// 4. script.js importa e inicializa.
// 5. localStorage guarda estado si el usuario debe conservar datos.
// 6. @media adapta el diseno a celular.

import { TodoList, DragSorter, TemperatureTool, BodyClassSwitcher, VisualReset } from './funciones.js';

new TodoList({ formId: 'todoForm', inputId: 'todoInput', listId: 'todoList', storageKey: 'project-todos'
});
new DragSorter('todoList', (order) => console.log(order));
new TemperatureTool({
  buttonSelector: '.unit-button',
  inputId: 'temperatureInput',
  calculateId: 'temperatureCalculate',
  messageId: 'temperatureMessage',
  resultId: 'temperatureResult'
});
new BodyClassSwitcher('[data-bg-class]', ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin']);
new VisualReset({ buttonId: 'resetVisual', classesToRemove: ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin'] });
```

Checklist final de integracion

- Cada id del HTML existe una sola vez.
- Cada clase usada por JavaScript existe en CSS si necesita estilo visual.
- Cada data-attribute tiene el mismo nombre que lee JavaScript.
- Las claves de localStorage son unicas por proyecto.
- No mezclas import/export con script normal en el mismo flujo.
- La pagina se prueba en desktop, tablet y celular.
- La consola del navegador no muestra errores.

Actualizacion del bloque 41 a 54

Este anexo conserva el PDF original 41 a 50 y agrega solo las cuatro funcionalidades nuevas o ajustadas para que la numeracion del proyecto quede alineada hasta la 54.

51. Version normal sin modulos

Permite usar JavaScript en un solo archivo sin import y export. Sirve en ejercicios academicos, demos sencillas y entregas rapidas donde no necesitas modularizar.

Para reutilizarlo: crea script-normal.js, pega helpers y clases en el orden correcto, cargalo al final del body y evita mezclarlo con imports.

Checklist rapido:

- No dejes export o import en el archivo.
- Carga primero helpers y luego inicializacion.
- Verifica que los IDs del HTML existan antes de ejecutar el script.

52. Responsive design reutilizable

Hace que la pagina se adapte a movil, tablet y escritorio. Ajusta grid, padding, radios y tipografia segun el ancho disponible.

Para reutilizarlo: usa contenedores con width min(...), convierte layouts de varias columnas a una sola en tablet y movil, y prueba en menos de 520px.

Checklist rapido:

- No debe aparecer scroll horizontal.
- Las cards deben pasar a una columna en pantallas pequenas.
- Navbar, tablas y formularios deben seguir legibles.

53. Tarjeta de presentacion Bootstrap

Actividad guiada basada en LAB1. La meta es construir una card personal con avatar, nombre, rol, badges y enlaces, manteniendo la logica de Bootstrap pero con el estilo de este proyecto.

Paso a paso: crea el contenedor principal, arma la card frontal, agrega la vista de tecnologias, aplica estilos tipo card y conecta un boton para alternar el estado visual.

Puntos clave:

- Reutiliza ideas como card, badge, btn y grid.
- Separa estructura HTML, look visual y cambio de estado.
- Verifica legibilidad en tema claro y oscuro.

54. Carrito de compras DOM

Actividad guiada para practicar DOM: agregar productos, actualizar badge, calcular total, eliminar items y vaciar el carrito sin recargar la pagina.

Paso a paso: selecciona los botones de producto, lee data-name y data-price, crea items dinamicos con createElement, recalcula total y conecta vaciado general.

Puntos clave:

- Convierte precios a numero antes de sumarlos.
- Actualiza badge y total despues de cada accion.
- Prueba agregar, eliminar y vaciar para validar consistencia.